

Examen-SO-Modulo-II-2019-D3.pdf



Darigod



Sistemas Operativos



2º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada**



[Accede al documento original](#)



Escuela de
Organización
Industrial

Contigo que evolucionas.
Contigo que lideras. Contigo que transformas.

**Esto es EOI.
Mismo propósito,
nueva energía.**



Descubre más aquí



EOI Escuela de
Organización
Industrial

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

Prácticas de Sistemas Operativos

Sistemas Operativos - Grupo D3- Grado en Informática

Prueba Módulo 2 de Prácticas – 18/12/2019

Apellidos-Nombre, y DNI:

NORMAS

- Acceda a su cuenta personal especificando el código que indicará El profesor.
- Hay que entregar esta hoja incluyendo los Apellidos-Nombre y DNI.
- No está permitido utilizar ningún dispositivo (calculadora, móviles, etc).
- Sólo se accederá a Internet a Prado para entregar las soluciones; suba los archivos con código fuente incluyendo los comentarios que considere oportunos como explicación.

Importante:

- Use llamadas al sistema y realice las comprobaciones y actuaciones adecuadas ante posibles errores.
- Los programas deben contener únicamente lo que se pide, se penalizará si incluyen funcionalidad no requerida, y es importante que funcionen.

Ejercicios

1) [4 puntos] Desarrolle un programa C denominado **duplicar** que utilice llamadas al sistema y haciendo uso de los mecanismos que considere oportunos, que lleve a cabo el duplicado exacto pero únicamente de la estructura de directorios contenidos en un directorio origen que se especificará como primer argumento dentro de otro directorio destino que se especificará como segundo argumento. Si el directorio destino no existe entonces se creará, los nombres de todos los directorios a crear tendrán el mismo nombre y permisos que tienen los directorios origen, y finalmente enviará a su salida estándar el número total de directorios que ha duplicado. El formato de ejecución será el siguiente: **duplicar** <directorio-fuente> <directorio-destino>

2) Cree otra versión del programa anterior que se ejecutará con el formato:

duplicar2 <directorio-fuente1> <directorio-fuente2> ... <directorio-destino>

a) [3 puntos] El programa **duplicar2** admitirá más de un directorio fuente, aunque sólo admitirá un directorio destino. Este programa **duplicar2** creará, para cada una de las parejas directorio fuente y directorio destino, un proceso hijo que llevará a cabo la ejecución del programa **duplicar** para dicha pareja. El programa **duplicar2** terminará cuando acaben todos sus procesos hijos, y cada vez que se detecte la terminación de un hijo imprimirá un mensaje en pantalla informando de ello.

b) [3 puntos] Adicionalmente al apartado b), al terminar el programa **duplicar2** será, en lugar de sus hijos, el único encargado de imprimir en pantalla el número total de directorios que han duplicado todos sus procesos hijos.

Examen SO Módulo II 2019

Ej 1

```
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <fcntl.h>
#include <dirent.h>
#include <sys/stat.h>
#include <time.h>
#include <errno.h>
#include <limits.h>

int total = 0;

int duplicar(const char *orig, const char *dest)
{
    DIR *orig_ptr = opendir(orig);
    if (orig_ptr == NULL) {
        perror("No existe el directorio de origen");
        exit(1);
    }

    DIR *dest_ptr = opendir(dest);
    if (dest_ptr == NULL) {
        // No existe: lo creamos
        if (mkdir(dest, 0777) < 0) {
            perror("mkdir destino");
            exit(1);
        }
    }
}
```

```

} else {
    // Existe: comprobamos que sea directorio
    struct stat st_aux;
    lstat(dest, &st_aux);
    if (!S_ISDIR(st_aux.st_mode)) {
        perror("Existe pero NO es un directorio");
        exit(1);
    }
    closedir(dest_ptr);
}

struct dirent *dir;

while ((dir = readdir(orig_ptr)) != NULL) {

    char *name = dir->d_name;

    /* Saltamos . y .. */
    if (strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
        continue;

    char o_path[PATH_MAX];
    char d_path[PATH_MAX];

    snprintf(o_path, sizeof(o_path), "%s/%s", orig, name);
    snprintf(d_path, sizeof(d_path), "%s/%s", dest, name);

    struct stat st;
    lstat(o_path, &st);

    if (S_ISDIR(st.st_mode)) {
        mkdir(d_path, st.st_mode & 0777); // umask=0 → permisos exactos
        total++;
        duplicar(o_path, d_path);
    }
}

closedir(orig_ptr);

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
return 0;
}

int main(int argc, char *argv[])
{
    if (argc != 3) {
        fprintf(stderr, "Uso: %s <dir_orig> <dir_dst>\n", argv[0]);
        return 1;
    }

    char *orig=argv[1];
    char *dest=argv[2];
    umask(0); // Eliminamos umask para conservar permisos

    duplicar(orig, dest);

    printf("%d\n", total);
    return 0;
}
```

Ej2

```
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <fcntl.h>
#include <dirent.h>
#include <sys/stat.h>
#include <time.h>
#include <errno.h>
#include <limits.h>
```

```

int main(int argc, char *argv[]){
    if( argc < 3){
        perror("./duplicar2 <orig> ... <dest>");
    }

    int n_hijos= argc -2;
    char *dest = argv[argc - 1];    // último argumento

    int pipes[n_hijos][2];
    pid_t pids[n_hijos];

    //Creamos los hijos
    for(int i=0; i < n_hijos; i++){
        if (pipe(pipes[i]) < 0) { //Creamos pipe
            perror("pipe");
            return 1;
        }
        pid_t pid=fork();

        if( pid == 0){ //Hijo
            close(pipes[i][0]); //Hijos no leen
            if(dup2[(pipes[i][1], STDOUT_FILENO) < 0]){
                perror("dup2");
                exit(1);
            }
            close(pipes[i][1]); //Ya tenemos STDOUT

            //Cerramos el resto de pipes
            for (int k = 0; k < n_hijos; k++) {
                if (k != i) {
                    close(pipes[k][0]);
                    close(pipes[k][1]);
                }
            }

            execlp("./duplicar", "duplicar", argv[i+1], dest, (char*)NULL); //Ejecutamos
        }
    }
}

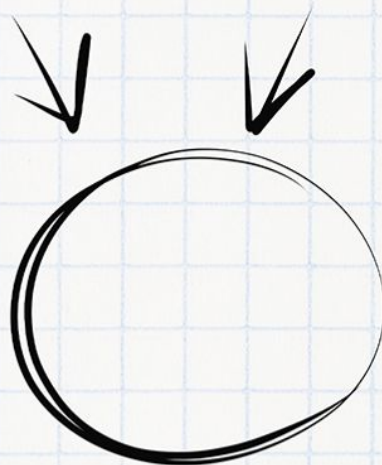
```


Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



WUOLAH

```

        perror("Error exec");
        exit(1);
    }
    else if(pid > 0){ //Padre
        pids[i]=pid; //Guardamos pid
    }
    else{
        perror("fork");
    }
}

//Padre espera a cualquier hijo
int i=0;
int total=0;

for (int i = 0; i < n_hijos; i++) {
    close(pipes[i][1]); // el padre nunca escribe
}

while( i < n_hijos){
    pid_t term=wait(NULL);
    printf("Hijo: %d TERMINADO\n", term);

    //Buscamos su pipe
    int pipe_index=-1;
    for(int i=0; i < n_hijos; i++){
        if(pids[i]== term){
            pipe_index=i;
        }
    }

    if(pipe_index==-1){
        perror("Error en la busqueda!!!");
        exit(1);
    }

    //Leemos el dato
    char buffer[64];
    int n;

```


Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH



```
if ( ( n=read(pipes[pipe_index][0],buffer, sizeof(buffer) -1)) < 0){  
    perror("Read");  
}  
buffer[n]= '\0';  
int dato = atoi(buffer);  
total += dato;  
close(pipes[pipe_index][0]); //Ya no lo usamos  
i++; //Un hijo menos  
}  
  
printf("Total de directorios: %d\n", total);  
  
}
```